THE FORGE

CFJ Management System

Work Integrated Learning - Task 2 Report

Mobile application, website, API and Oracle database

XISD6329 Work Integrated Learning (WIL) - POE Part 2, Task 2 (50%)

Module	XISD6329 Work Integrated Learning (WIL) - POE Part 2, Task 2 (50%)
Report date	19 October 2026
Repository	https://github.com/Manqoba275/CFJ-Management-System
Implementation commit	12c0312 (tag task2-submission)
Hosted website	https://gym-management-system-11sn.onrender.com/

Group members

Student name	Student number	Contribution
Sanele Manqoba Mazibuko	ST10444275	Architecture, API, Android, CI, report
Nonhlanhla Chirwa	ST10451192	Testing, acceptance, video plan
Tshifhiwa Thamagane	ST10441732	Database design, seeds, constraints
Nyito Ramudzuli	ST10450776	Requirements, business analysis, documentation

Contents

1 Purpose and context 5
1.1 Purpose of this report 5
1.2 The organisation and the problem 5
1.3 Target users 5
1.4 System architecture 5
1.5 Scope and status summary 7
2 Implementation against the Task 2 rubric 8
2.1 The app runs on a mobile device (10 marks) 8
2.2 Sign-in: registration, login, password security and SSO (10 marks) 8
2.3 Settings menu that makes sense for the application (10 marks) 8
2.4 Creation and use of the API (10 marks) 9
2.5 Integration of the API in the app (10 marks) 9
2.6 Database structure and table design (15 marks) 9
2.7 User interface (10 marks) 10
2.8 User documentation (10 marks) 10
2.9 GitHub, README and automated testing (10 marks) 10
2.10 Demonstration video (10 marks) 10
2.11 Rubric evidence summary 11
3 Database design and implementation 12
3.1 Platform and schema organisation 12
3.2 The fourteen tables 12
3.3 Constraints, keys and data types 13
3.4 Seed data 13
3.5 Verification evidence 14
3.6 Note on the alternative design 16
4 Testing and continuous integration 17
4.1 Test inventory 17
4.2 Web and API tests 17
4.3 Authentication-boundary tests 17
4.4 Android unit tests 17
4.5 Website structural checks 17
4.6 GitHub Actions 17
4.7 Coverage gaps 18
5 Evidence and screenshots 19
6 GitHub repository and GitHub Actions 26
6.1 Repository 26
6.2 Branching and pull requests 26
6.3 Workflows 26
6.4 Commit history 26
7 Team contribution 26
7.1 Method 26
7.2 Contribution matrix 26
7.3 Notes on attribution 27
8 Limitations and unproven items 28
9 Demonstration video 29
10 Conclusion 29
Appendix A Official Task 2 assessment rubric 30
Appendix B Evidence index 30

List of figures

Figure 1. System architecture: website and Android app authenticate through Firebase Authentication, call the Node.js API with a Firebase ID token, and the API is the only component that reads and writes Oracle. 6

Figure 2. Relational overview of the fourteen cfj_ tables: identity and profile, membership and training, operations, and community. 13

Figure 3. Captured Oracle query output: table row counts, a member profile record and constraint status for the cfj_ tables (18 October 2026 run, retained as evidence). 15

Figure 4. Schema verification report: fourteen tables, ten or more records each, and the named constraint checks reported as passing. 16

Figure 5. Website home page, live on Render at https://gym-management-system-11sn.onrender.com/. Public visitor view; the account page and the help system are reachable from the site navigation and footer. 19

Figure 6. Firebase account page on the website, showing the email/password fields and the Google sign-in option before a sign-in attempt (19 October 2026). It proves the page and Firebase SDK load; it does not prove a completed sign-in. 20

Figure 7. Google account-chooser hand-off on the website (19 October 2026). The flow was cancelled at the consent step, so this evidences the SSO hand-off, not a completed Google sign-in. 21

Figure 8. Invalid email input rejected with a readable message instead of a crash (18 October 2026). Evidence for the input-handling requirement. 21

Figure 9. Android app on a physical Huawei phone, 19 October 2026: the CFJ account screen shows 'Email verified. Signed in.', and the device notice confirms the phone has no Google Play services. The screenshot is user-supplied and is not bound to a build hash or install log. 22

Figure 10. Android unit test report: 16 tests, 0 failures, 100% successful (18 October 2026). 22

Figure 11. Android lint report: 0 errors and 10 warnings at the 18 October run; the 19 October build reports 0 errors and 22 warnings. Warnings are tracked as polish items, not failures. 23

Figure 12. GitHub Actions: Android build and tests workflow succeeded on the submission commit 12c0312 (run 39309331362). 23

Figure 13. GitHub Actions: setup checks workflow succeeded, running npm test and the website check (run 39309331410). 24

Figure 14. Help system page, live on Render. Part of the user-support requirement and linked from the footer of every page. 24

Figure 15. Booking and API integration evidence from the browser: a booking action against the local API, with the resulting state visible to a second client (19 October 2026). 25

List of tables

Table 1. Task 2 scope, what was delivered, and the evidence status at 19 October 2026. 7
Table 2. API implemented and exercised at commit 12c0312. 9
Table 3. Implementation status against the ten Task 2 marking criteria (100 marks). 11
Table 4. The fourteen cfj_ tables, their purpose and seeded row counts (verified 19 October 2026). 12
Table 5. Seeded row counts per table - 160 rows, minimum ten per table (19 October 2026). 14
Table 6. Automated and scripted checks run against the submission. 17
Table 7. Continuous integration runs cited in this report. 18
Table 8. Commit contribution by member, counted from all git refs on 19 October 2026. 26
Table 9. Items that are not proven, and their effect on the submission. 28
Table 10. Group members and student numbers. 30
Table 11. Evidence index: artefact, location and the claim it supports. 30

1 Purpose and context

1.1 Purpose of this report

This report accounts for the Task 2 deliverable of the XISD6329 Work Integrated Learning module: a working mobile application for the CFJ Management System, an updated website, a server-side API and an Oracle database. It describes what was built, how each Task 2 marking criterion is addressed, the evidence that supports the claims, the team contribution, and the items that remain unproven at the date of submission.

The report is written for the marker. Every technical claim is tied to a repository path, a captured artefact or a CI run so that it can be checked independently. Where evidence is incomplete, the status is stated in the relevant section and collected in Section 8 rather than presented as complete.

Repository: https://github.com/Mangoba275/CFJ-Management-System (public). Implementation commit: 12c0312. The commit is also tagged task2-submission on the default branch. The Render-hosted website is https://gym-management-system-11sn.onrender.com/.

1.2 The organisation and the problem

CFJ Lifestyle Fitness is a strength and conditioning gym in Bedfordview, Johannesburg. Its public site advertises CrossFit, HYROX, Concept Cardio, specialty programmes, a marathon event and a session-based membership model (three, four or unlimited sessions per week, drop-ins and ten-, fifteen- and twenty-session concessions). Timetable bookings are handled by a third-party product (Octiv).

The administrative problem is fragmentation: member details, membership validity, class bookings, attendance, simulated payments, trainer requests and community activity are recorded in separate places, and the website and any mobile client do not share one source of truth. The result is duplicated capture, booking disputes when a class fills up, and no reliable view of who attended what.

The CFJ Management System addresses this with one relational model behind one API. Both clients - the website and the Android app - authenticate through the same Firebase project, send a Firebase ID token to the same Node.js API, and are served from the same Oracle tables. Class capacity, booking ownership, attendance and payment retries are enforced server-side in a single transaction so that two clients cannot oversell a class.

This is a student project. CFJ has not agreed to replace Octiv or to open its private systems to the team, so no live customer data, no real payments and no live business integration form part of the implementation. All member records used for demonstration are synthetic.

1.3 Target users

- Members - register and sign in, keep a display name and training goal, view the timetable, book and cancel classes, see attendance and payment history, join community events and message other members.
- Prospective members - read programmes and membership options on the website, see the timetable, and send an introductory enquiry.
- Trainers and front desk - record attendance, resolve booking and payment retries, and manage class capacity. These operations are exercised by the API and its test suites; they are not exposed as a finished Android screen.
- Management - consume the computed reports endpoint. The active schema models members, operations and community. Administrative roles are enforced server-side; there is no staff or role table in the active schema, which is a deliberate scope decision recorded in Section 8.

1.4 System architecture

The system has four layers. The website (HTML, CSS and JavaScript) and the Android app (Kotlin, native Android views) are the clients. Firebase Authentication is the identity service and holds the credentials. The Node.js API is the only component that talks to the database: it verifies the Firebase ID token, resolves the Firebase UID to a member record, applies business rules and runs the Oracle statements. Oracle Database Free (FREEPDB1) is the relational store, holding fourteen cfj_ prefixed tables.

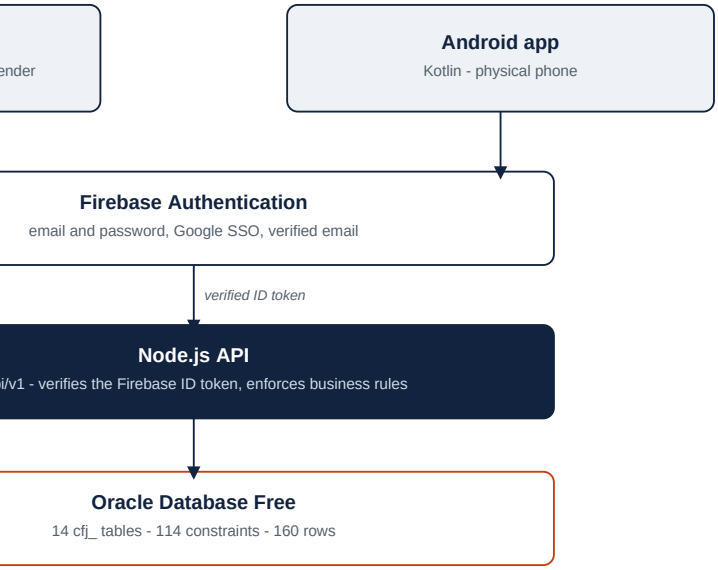


Figure 1. System architecture: website and Android app authenticate through Firebase Authentication, call the Node.js API with a Firebase ID token, and the API is the only component that reads and writes Oracle.

Passwords are never stored by application code. The Android account screen clears the password field after each attempt and never logs it; Firebase performs hashing and transport over HTTPS. Oracle stores business records only - no password and no credential. The API refuses a request without a verified email and an existing identity mapping, so a browser cannot assert a member ID or a role (backend/firebase-gate.mjs, backend/oracle-identity-store.mjs).

Hosting reality at submission: the website is live on Render (HTTP 200, deployed from the project repository). The API and Oracle instance run on the project computer and are bound to loopback, so they are reachable by the USB-connected phone and the local browser but not from the public internet.

That limitation is stated again in Section 8.

1.5 Scope and status summary

Table 1. Task 2 scope, what was delivered, and the evidence status at 19 October 2026.

Area	Delivered	Status
Website	Public site (17 pages), Firebase account page, help system, Render deployment	Live and verified
Android app	Kotlin app: training dashboard, classes, settings, account, debug API panels	Builds; runs on a physical phone (one screenshot)
Authentication	Email/password register, verify, reset, sign out; Google SSO code on both clients	Implemented; signed-in state observed once; SSO end-to-end unproven
API	Node.js HTTP API: profile, operations, bookings, community, management, Firebase gate	Implemented; 51 automated tests pass; not hosted
Database	Oracle FREEPDB1, 14 cfj_ tables, 114 constraints, 160 seeded rows	Verified 19 Oct 2026 (local instance)
CI	GitHub Actions: Android build/tests/lint, setup checks	Both workflows green on the implementation commit
User documentation	User guide, live help page, Android account documentation	Present; Android account and training sections updated 19 October 2026
Demonstration video	Narrated physical-phone video	Not yet recorded - see Section 9

2 Implementation against the Task 2 rubric

The Task 2 assessment sheet awards 100 marks across ten criteria. This section takes each criterion in the order of the official rubric, states what was implemented, cites the evidence, and gives an honest status. Section 2.11 collects the statuses in one table. The official assessment sheet is appended to this report as Appendix A.

2.1 The app runs on a mobile device (10 marks)

The Android client is a native Kotlin application (package za.co.cfjlifestylefitness.app). It is built with Gradle 9.4.1 and Android Gradle Plugin 9.2.1, compiles and targets SDK 36 and supports Android 8.0 (API 26) and later. The debug APK in the submission pack is 8,445,659 bytes with SHA-256 90A810FF9248974A06E3D6BA570354E962912B398FB6F7EEBF9067835A725E76, built from commit 78a36ff; the source code is unchanged in 12c0312 (BUILD SUCCESSFUL in 7m 52s on re-verification, including unit tests and lint).

The app was run on a physical Huawei phone, not an emulator. Figure 9 is a phone screenshot (421 x 904 pixels, Android status bar with carrier, battery and signal) showing the CFJ account screen with the message 'Email verified. Signed in.' and the device-specific notice 'This phone has no Google Play services'. That message is produced by AccountActivity only when the app detects a device without Play services, which is positive evidence that this build ran on that phone.

Status: partially proven. The screenshot shows the app running on a physical phone and a verified signed-in state. It is not tied to a build hash, device model or install log, and team records note that a later install attempt found the phone offline. The claim that the APK in the pack is the exact build shown on screen is therefore not made; the installation record is listed as outstanding in Section 8.

2.2 Sign-in: registration, login, password security and SSO (10 marks)

The account screen (AccountActivity.kt) provides account creation, email/password sign-in, email verification (send and re-check), password reset and sign-out. Firebase handles credential storage and transport: the app never writes a password to SharedPreferences, Oracle or the log, and the password field is cleared after every attempt. Firebase error codes are mapped to readable messages and the email pattern and non-empty password are validated before any network call, so invalid input does not crash the app. The class comment records this behaviour.

Single sign-on is implemented on both clients. On Android, Google sign-in uses the Credential Manager account chooser and exchanges the Google ID token for a Firebase session (GoogleAccount.kt). On the website, the Firebase JavaScript SDK offers a Google popup from the account page (website/account.html, website/firebase-account.js). The Android OAuth client for the debug signing certificate is registered in google-services.json (SHA-1 16:5B:BA:2B:C9:28:86:FD:8A:F6:9F:5C:70:D0:8F:02:30:23:E0:EF).

Status: implemented, partially verified. Figure 9 shows a verified signed-in state on the phone. A register - sign out - sign in again cycle has not been captured as a separate artefact, Google consent was not completed end-to-end (Figure 7 shows the account chooser hand-off that was then cancelled), and the demonstration phone has no Google Play services, so Google SSO cannot complete on that device. SSO is proven as code and configuration, not as a completed sign-in.

2.3 Settings menu that makes sense for the application (10 marks)

Settings (screen_settings.xml, MainActivity.kt) captures a display name (validated to 2-60 characters) and a fitness goal from a fixed list, with explicit Save and a Reset that asks for confirmation. Values persist in the cfj_preview SharedPreferences file and are restored when the screen reopens. The screen also exposes the CFJ account entry point, which opens the sign-in/registration screen, and - in debug builds - the profile sync and community panels.

Status: implemented with known gaps. There is no notification, privacy or account-management settings (change password is available from the account screen; account deletion is not offered), the saved display name and goal are not displayed on the new training dashboard, and one string resource still states that no member account is created. These are user-facing polish items, not functional failures, and are listed in Section 8.

2.4 Creation and use of the API (10 marks)

The API is a Node.js HTTP service in backend/. It is versioned under /api/v1, takes a Bearer token, validates input, enforces ownership and returns explicit status codes: 401 for a missing or invalid identity, 403 for a denied permission, 409 for capacity, duplicate or version conflicts, and 400 for invalid input. Server modules cover profiles, operations (bookings, attendance, payments, reports), community features and the Firebase gate that verifies ID tokens and maps the UID to cfj_members.

Table 2. API implemented and exercised at commit 12c0312.

Capability	Endpoint(s)	Server module
Member profile	GET /api/v1/me, PATCH /api/v1/me	profile-server.mjs, firebase-oracle-server.mjs
Classes	GET /api/v1/classes	operations.mjs
Bookings	POST /api/v1/bookings, DELETE /api/v1/bookings/{id}	operations.mjs
Attendance and payments	POST /api/v1/attendance, GET /api/v1/me/payments, POST /api/v1/payments	operations.mjs
Community	Events, registrations, resources, messages, friend connections, trainer requests	community.mjs
Management reports	GET /api/v1/admin/reports	management.mjs
Identity	Firebase ID token verification, UID to member resolution	firebase-gate.mjs, oracle-identity-store.mjs

The API is exercised by 51 automated tests (Section 4), including six authentication-boundary tests that prove unsigned requests, unverified emails and unmapped identities are rejected. The Firebase listener is a separate entry point (backend/firebase-oracle-server.mjs) that requires a verified email and an existing identity mapping before it will serve a request.

Status: created and used, tested locally. Two gaps are recorded honestly. First, the API is not hosted: it binds to loopback on the project computer. Second, there is no OpenAPI contract file yet, although backend/README.md asks for one. Neither gap affects the correctness of the implemented routes.

2.5 Integration of the API in the app (10 marks)

The Android app calls the API rather than the database. ProfileApi performs GET and PATCH /api/v1/me with a Bearer token and a JSON body; CommunityApi calls the community endpoints; FirebaseAccess resolves a Firebase UID to a live ID token on a background thread and parses the response; RemoteProfile rejects coerced, fractional, negative or stale version values before they are used. Class responses follow a contract shared by the website and the Android model (id, name, trainer, spaces), so the same payload renders on both clients.

Status: implemented, but deliberately gated to debug builds. The sync and community panels are only bound when BuildConfig.DEBUG is true, so a release build shows authentication but no visible API integration. A live Firebase token to API to Oracle round trip recorded from the app is not in the evidence pack; the API side of that path is proven by the auth-boundary and operations tests, but the end-to-end device-to-Oracle demonstration is outstanding (Section 8).

2.6 Database structure and table design (15 marks)

Fourteen cfj_ tables are implemented in Oracle Database Free with full primary and foreign keys, unique constraints, check constraints and consistent naming. The design is verified by an automated schema verifier that reports 21 named checks passing, 114 constraints and 160 seeded rows. Section 3 describes the schema, the constraints and the verification in detail, and Figure 2 shows the relationships.

2.7 User interface (10 marks)

The app has five screens: a training dashboard (Home) with three illustrated exercise guides, set completion and a rest timer; Classes, a timetable view with capacity indication; Settings; the CFJ account screen; and a local completed-session counter. Navigation uses a bottom bar with a selected state and 52 dp touch targets, the exercise illustrations carry content descriptions, and status messages are live regions for accessibility. Invalid input is handled without crashing: the email pattern and password are validated before use, Firebase errors are mapped to readable text, and the booking and profile layers reject malformed or stale values with explicit messages.

Status: implemented; visual defects remain. The account screen builds its views in code with hardcoded strings rather than string resources, Classes is still a static timetable preview without booking, and a few exercise captions contain mojibake characters. Lint reports 0 errors and 22 warnings, including a missing accessibility label and borderless button styling. These are recorded in Section 8 rather than hidden.

2.8 User documentation (10 marks)

Documentation is delivered in three forms. The user guide (docs/user-guide/USER_GUIDE.md, also issued as an eight-page Word document) covers the website flows, the help system and the Android app: the CFJ account screen, registration, email verification, password reset, Google sign-in, the training dashboard and Settings. It was rewritten on 19 October 2026 to match the current app. The help page is part of the live website and is reachable from the footer of every page. The Firebase integration notes (docs/FIREBASE_INTEGRATION.md) document the account flow and the fact that passwords are not stored by the app.

Status: present and live. The Android section of the user guide documents the account, verification, reset, Google sign-in, training dashboard and Settings screens as they are in the submitted build, and the help system is live on the hosted site (Figure 14). Screenshots in the guide are browser captures; Android captures remain in the evidence pack.

2.9 GitHub, README and automated testing (10 marks)

The repository is public, has more than 190 workflow runs and a real commit history from four contributors. Two GitHub Actions workflows run on every push: Android build and tests (unit tests, lint, debug and androidTest APK assembly, signing checks, artifact upload) and setup checks (npm test and the 17-page website check). Both workflows are green on the implementation commit (Section 6). Automated test coverage is described in Section 4.

Status: meets the criterion in substance, with two repository hygiene problems stated plainly. The README on the default branch is stale - it still calls the Android app planned and lists sign-in as pending - and the Firebase work is on the feature branch merged through PR #6, which remains open for review at the time of writing. The commit cited in this report is the implementation head, and the report states the branch explicitly so the marker is not misled.

2.10 Demonstration video (10 marks)

The narrated, physical-phone demonstration video required by the rubric has not been recorded. A recording plan exists (docs/user-guide/DEMONSTRATION_PLAN.md) but no video file or link is in the repository or the submission pack. This is the one criterion with no evidence at all; Section 9 defines the

exact content the recording must cover.

2.11 Rubric evidence summary

Table 3. Implementation status against the ten Task 2 marking criteria (100 marks).

Criterion	Marks	Status	Primary evidence
App runs on a mobile device	10	Partial	Phone screenshot (Figure 9); APK build log; CI artifact
Sign in (register/login, encrypted password, SSO)	10	Implemented, partly proven	AccountActivity.kt; Firebase; Figure 9; Figure 7
Settings	10	Implemented	screen_settings.xml; MemberPreferencesTest (4 tests)
Creation/use of API	10	Implemented, local only	backend modules (Table 2); 51 tests; 6 auth-boundary tests
API integration in app	10	Implemented, debug-only	ProfileApi, CommunityApi, FirebaseAccess; contract tests
Database structure and table design	15	Implemented and verified	14 tables, 114 constraints, 160 rows, 21 checks (Section 3)
User interface	10	Implemented; polish gaps	Five screens; lint 0 errors / 22 warnings; Figure 9
User documentation	10	Present, partly stale	User guide; live help page (Figure 14); Firebase notes
GitHub, README and automated testing	10	Substantially met	Public repo; two workflows green; 191 runs; stale README
Demonstration video	10	Not recorded	Section 9

3 Database design and implementation

3.1 Platform and schema organisation

The database is Oracle Database Free (FREEPDB1), reached through node-oracledb from the API. Schema changes are applied as ordered, idempotent migrations in database/: 001_profiles.sql (identity and profile), 002_operations.sql (memberships, trainers, classes, bookings, attendance, payments), 003_community.sql (trainer requests, friend connections, events, event registrations, resources, messages) and 004_api_retries.sql (API retry and audit columns). Migrations 001-004 were re-applied on 19 October 2026 and are idempotent: a second run changes nothing.

All application tables use the cfj_ prefix, which separates the assessed schema from two older design proposals kept under database/design/alternative-proposal/ and explicitly marked as not applied. That separation prevents a reader from assessing tables that were never created.

3.2 The fourteen tables

Table 4. The fourteen cfj_ tables, their purpose and seeded row counts (verified 19 October 2026).

Table	Purpose	Rows
CFJ_MEMBERS	Member identity: member number, name, email, Firebase identity UID	20
CFJ_PROFILES	Display name, training goal, version for optimistic concurrency	20
CFJ_MEMBERSHIPS	Membership type, session allowance, validity dates	10
CFJ_TRAINERS	Trainer records used to staff classes	10
CFJ_CLASS_SESSIONS	Scheduled class, trainer, capacity, start and end time	10
CFJ_BOOKINGS	Member booking of a class session, with ownership and status	10
CFJ_ATTENDANCE	Check-in of a booked member at a session	10
CFJ_PAYMENTS	Simulated payment and retry record for a membership or booking	10
CFJ_TRAINER_REQUESTS	Member request for personal training	10
CFJ_FRIEND_CONNECTIONS	Canonical friend pair between two members	10
CFJ_EVENTS	Community event with date, capacity and consent rules	10
CFJ_EVENT_REGISTRATIONS	Member registration for an event	10
CFJ_RESOURCES	Training and nutrition resources with access level	10
CFJ_MESSAGES	Member-to-member or trainer message	10

The highest-value design decisions are the ownership keys. A booking carries the member and the session together, and the composite foreign key prevents a booking from being attached to another member's record. Friend connections are stored once per pair with an ordering check so the same friendship cannot exist twice in opposite directions. Event registrations use a unique key on event and email so a retry cannot double-register a member.

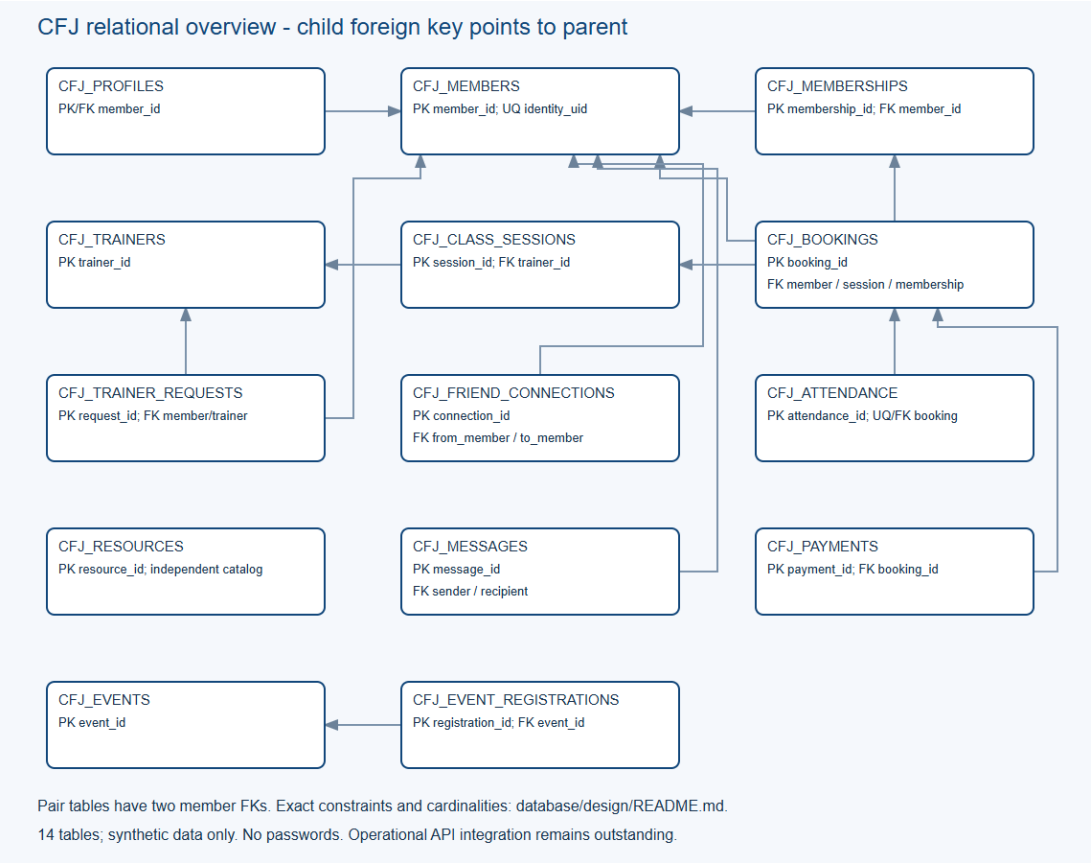


Figure 2. Relational overview of the fourteen cfj_ tables: identity and profile, membership and training, operations, and community.

3.3 Constraints, keys and data types

The schema verifier counts 114 constraints across the fourteen tables. Every table has a primary key; foreign keys cover every relationship; unique keys protect bookings, check-ins, retry idempotency, event email registrations and friend pairs; check constraints enumerate statuses, reject negative amounts and invalid capacities, enforce date ordering and accept fractional capacity and rating values. Instants use TIMESTAMP WITH TIME ZONE and membership validity uses DATE, so time-zone drift cannot change a booking day.

The 19 October re-verification reports every constraint as ENABLED and VALIDATED, which matters: Oracle will accept a disabled constraint silently, so the status is part of the proof that the rules are actually enforced.

3.4 Seed data

The database is populated with synthetic data only: example.invalid email addresses, SEED- identifiers and no real member information. Row counts are verified by query rather than by counting INSERT statements.

Table 5. Seeded row counts per table - 160 rows, minimum ten per table (19 October 2026).

Table group	Tables	Rows per table	Total
Identity	CFJ_MEMBERS, CFJ_PROFILES	20 each	40
Membership and training	CFJ_MEMBERSHIPS, CFJ_TRAINERS, CFJ_CLASS_SESSIONS	10 each	30
Operations	CFJ_BOOKINGS, CFJ_ATTENDANCE, CFJ_PAYMENTS	10 each	30
Trainer requests and friends	CFJ_TRAINER_REQUESTS, CFJ_FRIEND_CONNECTIONS	10 each	20
Events and resources	CFJ_EVENTS, CFJ_EVENT_REGISTRATIONS, CFJ_RESOURCES	10 each	30
Messaging	CFJ_MESSAGES	10 each	10
Total	14 tables	at least 10 per table	160

The seed script is re-runnable: a second execution inserts zero rows. On 19 October the seed run exited with status 1 by design. Member 9001 is bound to a 28-character external Firebase UID, and the seed script's reserved-fixture guard refuses to overwrite a bound identity. That guard is evidence that an external identity is attached to member 9001; it is not a seed failure, and the row counts above were verified after the refusal. A live token exchange against that binding remains unproven and is listed in Section 8.

3.5 Verification evidence

Database verification is produced by three scripts and stored as artefacts: verify-oracle-schema.mjs (14 tables, 160 rows, 21 named checks, constraint status), verify-oracle-operations.mjs (15 live operations checks against two independent HTTP servers) and the seed row-count report. The 19 October run reports PASS on the schema verification and PASS on the operations verification.

The 15 operations checks cover the behaviours that matter to the business: the last seat of a class cannot be sold twice, a repeated payment request is idempotent, attendance is recorded once per booking, management reports aggregate committed data, and event capacity is enforced. Test fixtures are cleaned up after the run.

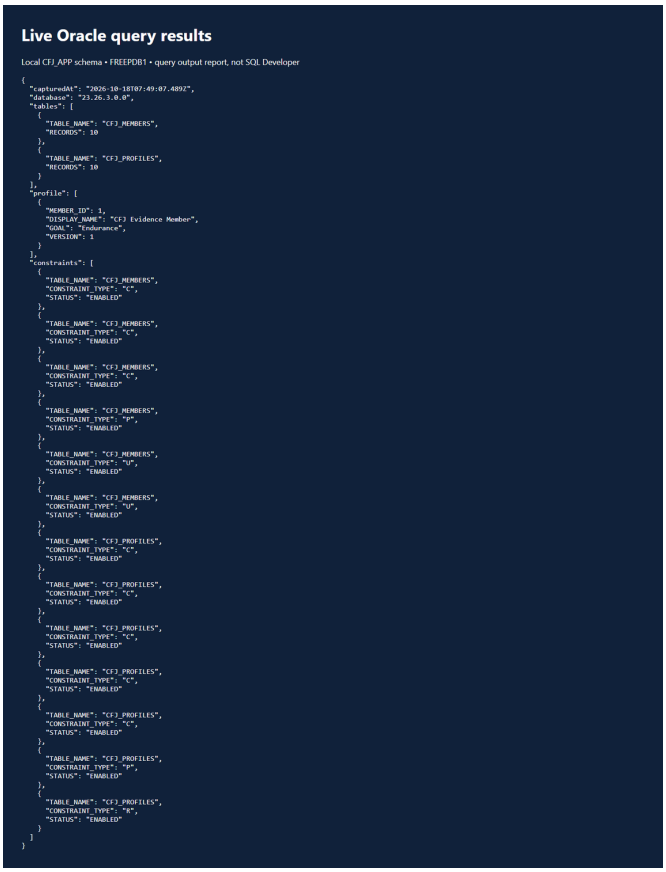
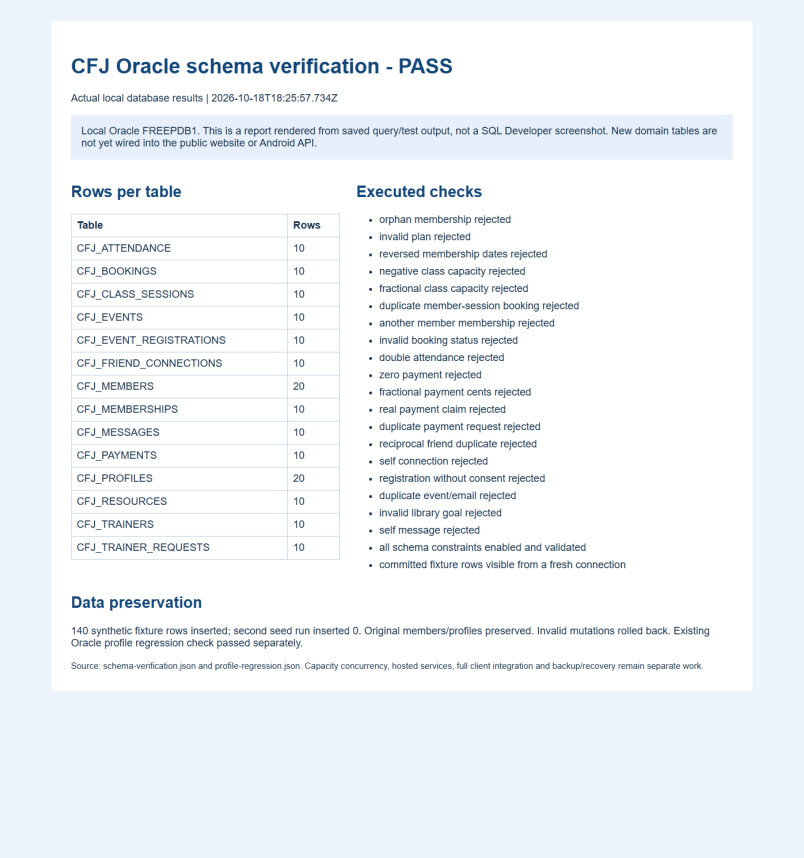


Figure 3. Captured Oracle query output: table row counts, a member profile record and constraint status for the cfj_ tables (18 October 2026 run, retained as evidence).



CFJ Oracle schema verification - PASS

Actual local database results | 2026-10-18T18:25:57.734Z

Local Oracle FREEPDB1. This is a report rendered from saved query/test output, not a SQL Developer screenshot. New domain tables are not yet wired into the public website or Android API.

Rows per table

Table	Rows
CFJ_ATTENDANCE	10
CFJ_BOOKINGS	10
CFJ_CLASS_SESSIONS	10
CFJ_EVENTS	10
CFJ_EVENT_REGISTRATIONS	10
CFJ_FRIEND_CONNECTIONS	10
CFJ_MEMBERS	20
CFJ_MEMBERSHIPS	10
CFJ_MESSAGES	10
CFJ_PAYMENTS	10
CFJ_PROFILES	20
CFJ_RESOURCES	10
CFJ_TRAINERS	10
CFJ_TRAINER_REQUESTS	10

Executed checks

- orphan membership rejected
- invalid plan rejected
- reversed membership dates rejected
- negative class capacity rejected
- fractional class capacity rejected
- duplicate member-session booking rejected
- another member membership rejected
- invalid booking status rejected
- double attendance rejected
- zero payment rejected
- fractional payment cents rejected
- real payment claim rejected
- duplicate payment request rejected
- reciprocal friend duplicate rejected
- self connection rejected
- registration without consent rejected
- duplicate event/email rejected
- invalid library goal rejected
- self message rejected
- all schema constraints enabled and validated
- committed fixture rows visible from a fresh connection

Data preservation

140 synthetic fixture rows inserted; second seed run inserted 0. Original members/profiles preserved. Invalid mutations rolled back. Existing Oracle profile regression check passed separately.

Source: schema-verification.json and profile-regression.json. Capacity concurrency, hosted services, full client integration and backup/recovery remain separate work.

Figure 4. Schema verification report: fourteen tables, ten or more records each, and the named constraint checks reported as passing.

Status: verified on a local Oracle instance. The database is not cloud-hosted, and the instance is not reachable after the demonstration session, so the check cannot be re-run by the marker from outside the project computer. The stored artefacts, the migration scripts and the verifier source are committed so the run can be reproduced on any Oracle Free instance.

3.6 Note on the alternative design

An earlier normalised design with fourteen unprefixed tables (app_user, member, booking and so on) is kept under database/design/alternative-proposal/. It is not applied to the database and does not feed the API. It is retained because it documents the design review, and it is labelled as a proposal so that no reader mistakes it for the assessed schema.

4 Testing and continuous integration

4.1 Test inventory

Table 6. Automated and scripted checks run against the submission.

Check	Result	What it proves
Web and API tests (npm test)	51 passed, 0 failed	Booking rules, capacity, retries, persistence, restart recovery, input validation
of which authentication-boundary tests	6 passed, 0 failed	Unsigned, unverified and unmapped identities are rejected by the Firebase gate
Android unit tests (testDebugUnitTest)	16 passed, 0 failed	Booking rules, preference validation, API contract and error mapping
Oracle schema verification	21 checks, PASS	14 tables, 160 rows, every constraint ENABLED and VALIDATED
Oracle operations verification	15 checks, PASS	Last-seat booking, idempotent payments, attendance, reports, event capacity
Website structural check (npm run check)	17 pages passed	Links, assets and script references across the site
Android lint	0 errors, 22 warnings	No lint errors; warnings tracked for polish

4.2 Web and API tests

The Node test suite runs 51 tests against the API and the website scripts. It covers booking eligibility and capacity, concurrent booking attempts for the last seat, cancellation and ownership, attendance and simulated payment retries, membership allowance across a week, stale-write rejection with optimistic versions, Unicode handling across network chunk boundaries, malformed JSON and oversized bodies, permission checks for private pages, and recovery after a restart. The suite is run by npm test and by the setup-checks workflow.

4.3 Authentication-boundary tests

Six tests exercise the Firebase gate directly. They prove that a request without a token, with an unverified email, with an unmapped UID, or with a token from a foreign origin is rejected, and that a mapped, verified identity is accepted. These are boundary tests on the server, not an Android sign-in test; the distinction matters and is repeated in Section 8.

4.4 Android unit tests

Sixteen unit tests run in the JVM on every Android build: BookingRulesTest (3), MemberPreferencesTest (4), ProfileApiTest (3), RemoteProfileTest (3), CommunityFailureTest (2) and SharedBookingContractTest (1). They cover eligibility decisions, preference and goal validation, profile API request and response handling, rejection of malformed or stale versions, and the shared class contract. They do not cover AccountActivity, GoogleAccount or FirebaseAccess, and no instrumentation test runs in CI - both gaps are recorded in 4.7.

4.5 Website structural checks

npm run check validates 17 pages for broken internal links, missing assets and script references that do not resolve. It is the check that keeps a static site coherent after navigation changes.

4.6 GitHub Actions

Two workflows run on every push and pull request. Android build and tests runs testDebugUnitTest, lintDebug, assembleDebug and assembleDebugAndroidTest, verifies the release signing configuration, and uploads the debug APK as a build artefact. Setup checks runs npm test and npm run check. Both

are green on the implementation commit:

Table 7. Continuous integration runs cited in this report.

Workflow	Run	Commit	Result
Android build and tests	39309331362	12c0312	success
Setup checks	39309331410	12c0312	success

Run URLs: https://github.com/Manqoba275/CFJ-Management-System/actions/runs/39309331362 and https://github.com/Manqoba275/CFJ-Management-System/actions/runs/39309331410. The workflows are described in Section 6.

4.7 Coverage gaps

Three gaps are stated rather than glossed over. The Android unit tests do not touch the five authentication classes; the instrumentation smoke test exists but CI never runs it, because it needs a device and a development token; and the Oracle adapters are covered by the standalone verifier scripts, not by the CI suite. Closing these is listed as follow-up work in Section 8.

5 Evidence and screenshots

Every screenshot in this section is labelled with what it shows, when it was captured and what it does not prove. Historical captures are dated; they are not presented as evidence of newer builds. No screenshot contains a password, token or personal information, and credential fields are not captured.

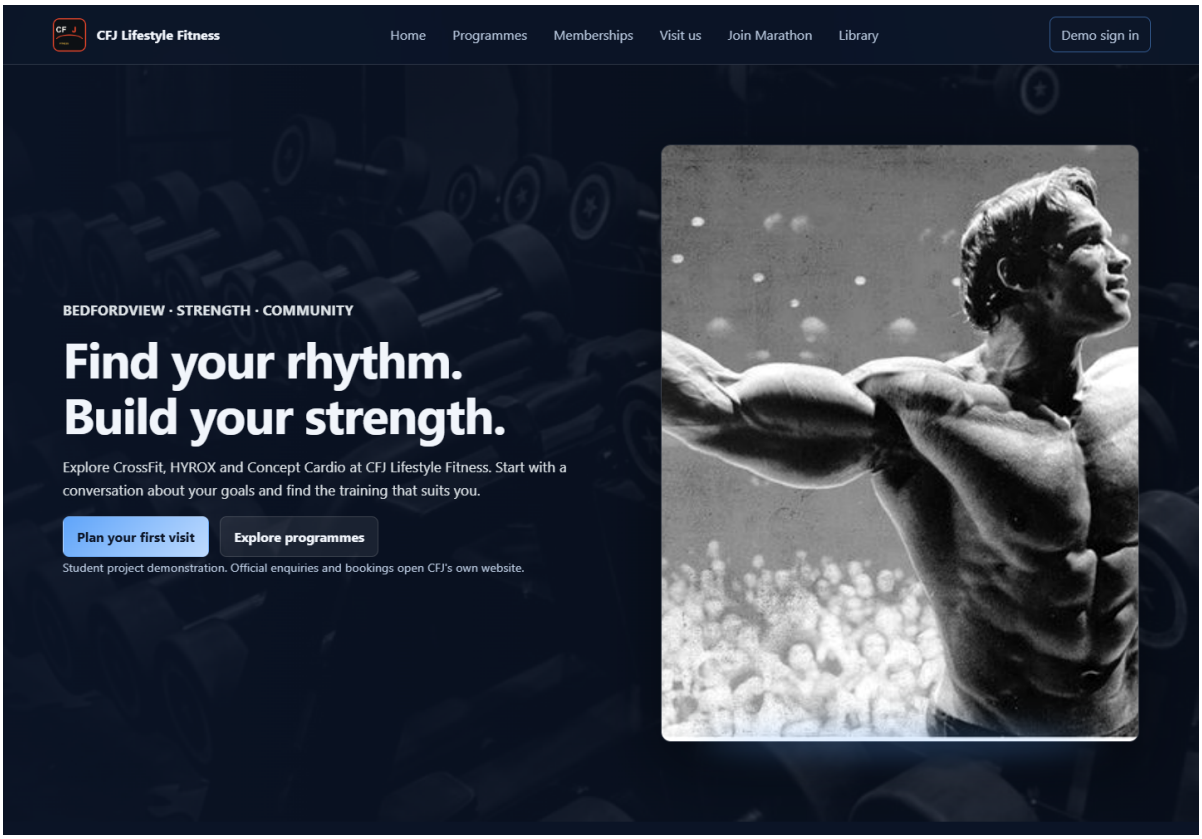


Figure 5. Website home page, live on Render at https://gym-management-system-11sn.onrender.com/. Public visitor view; the account page and the help system are reachable from the site navigation and footer.

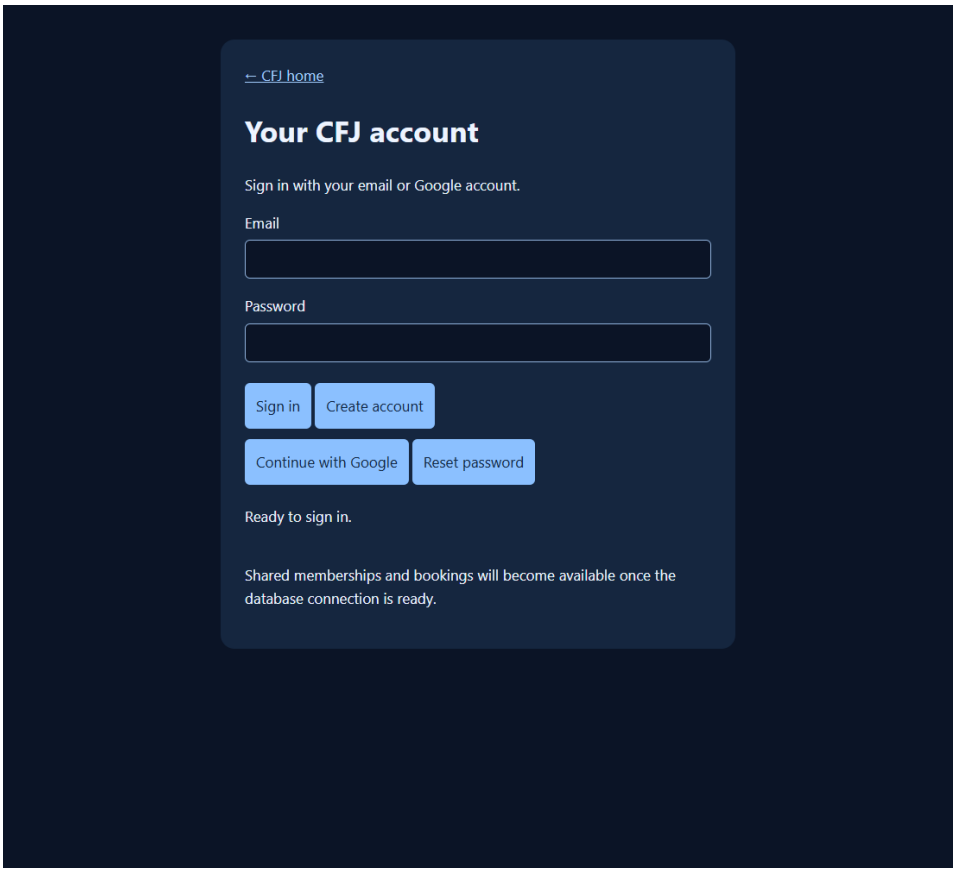


Figure 6. Firebase account page on the website, showing the email/password fields and the Google sign-in option before a sign-in attempt (19 October 2026). It proves the page and Firebase SDK load; it does not prove a completed sign-in.

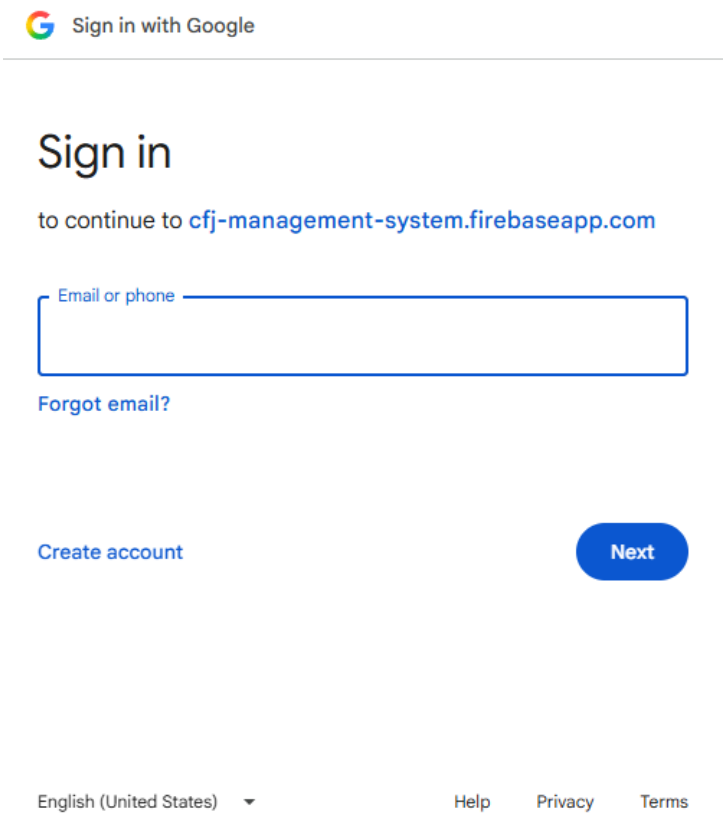


Figure 7. Google account-chooser hand-off on the website (19 October 2026). The flow was cancelled at the consent step, so this evidences the SSO hand-off, not a completed Google sign-in.

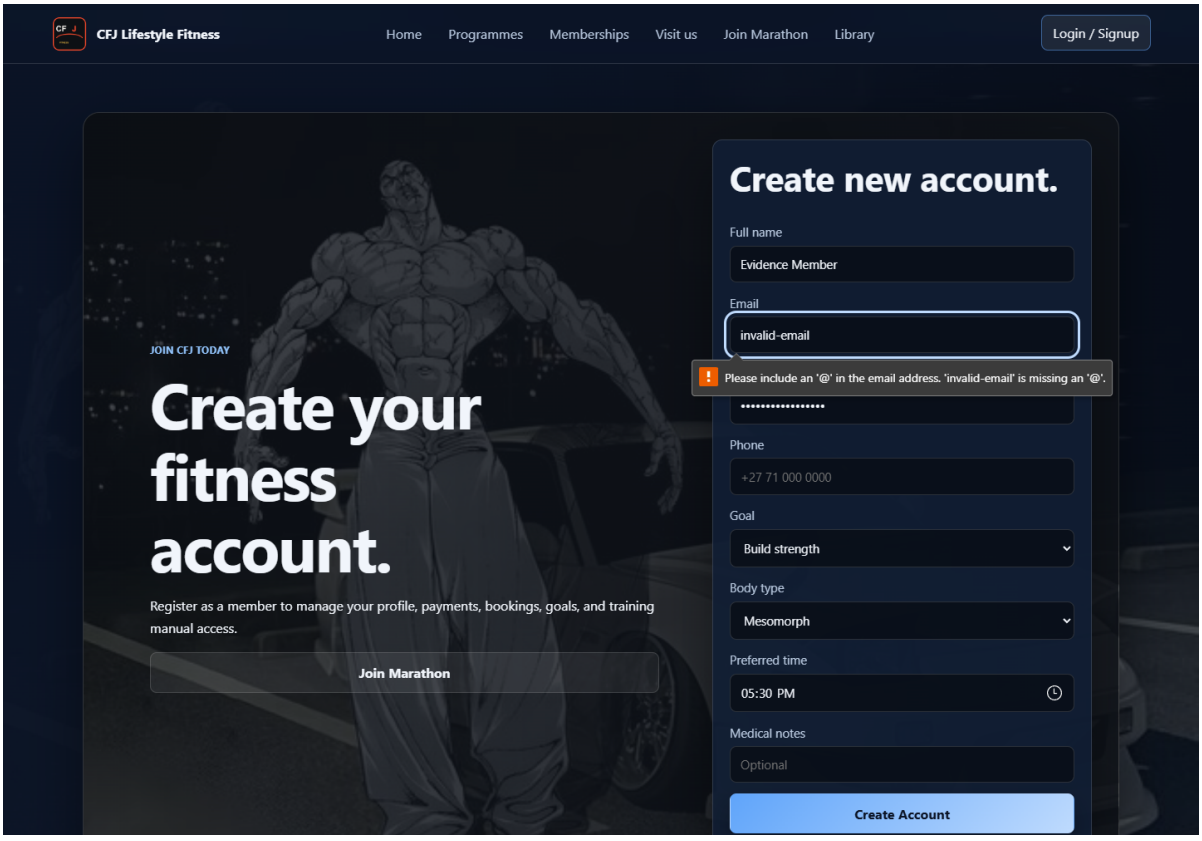


Figure 8. Invalid email input rejected with a readable message instead of a crash (18 October 2026). Evidence for the input-handling requirement.

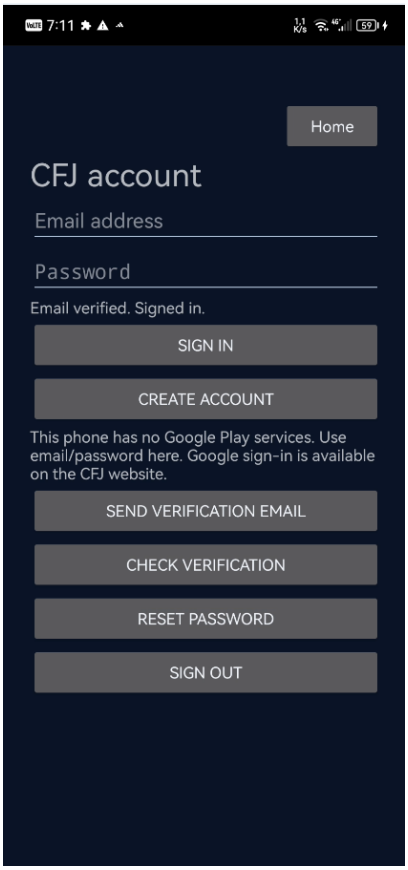
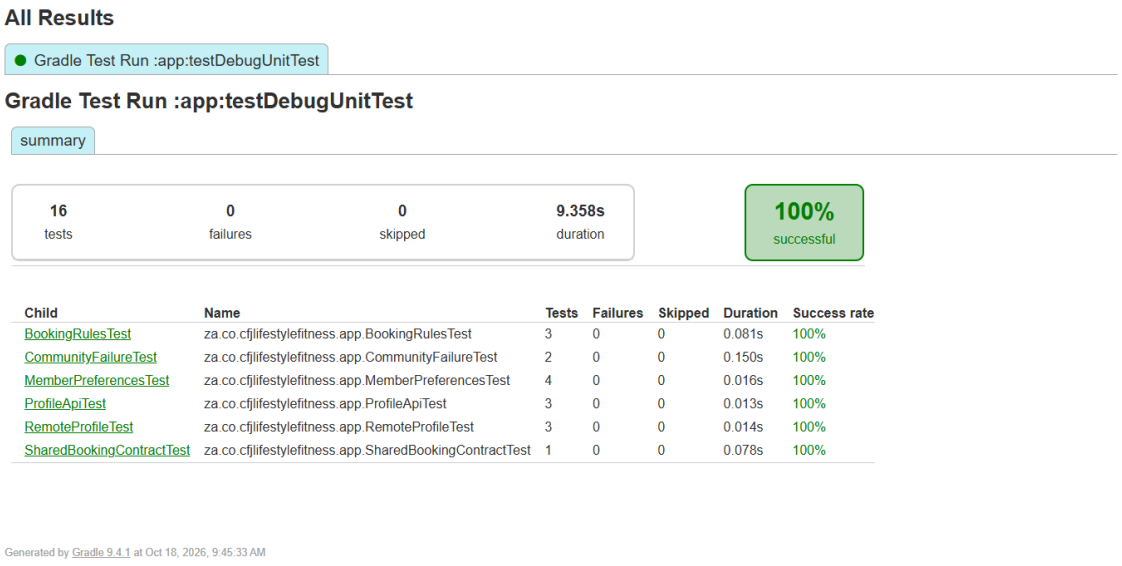


Figure 9. Android app on a physical Huawei phone, 19 October 2026: the CFJ account screen shows 'Email verified. Signed in.', and the device notice confirms the phone has no Google Play services. The screenshot is user-supplied and is not bound to a build hash or install log.



All Results

Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

16	0	0	9.358s	100%
tests	failures	skipped	duration	successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
BookingRulesTest	za.co.cfjlifestylefitness.app.BookingRulesTest	3	0	0	0.081s	100%
CommunityFailureTest	za.co.cfjlifestylefitness.app.CommunityFailureTest	2	0	0	0.150s	100%
MemberPreferencesTest	za.co.cfjlifestylefitness.app.MemberPreferencesTest	4	0	0	0.016s	100%
ProfileApiTest	za.co.cfjlifestylefitness.app.ProfileApiTest	3	0	0	0.013s	100%
RemoteProfileTest	za.co.cfjlifestylefitness.app.RemoteProfileTest	3	0	0	0.014s	100%
SharedBookingContractTest	za.co.cfjlifestylefitness.app.SharedBookingContractTest	1	0	0	0.078s	100%

Generated by Gradle 9.4.1 at Oct 18, 2026, 9:45:33 AM

Figure 10. Android unit test report: 16 tests, 0 failures, 100% successful (18 October 2026).

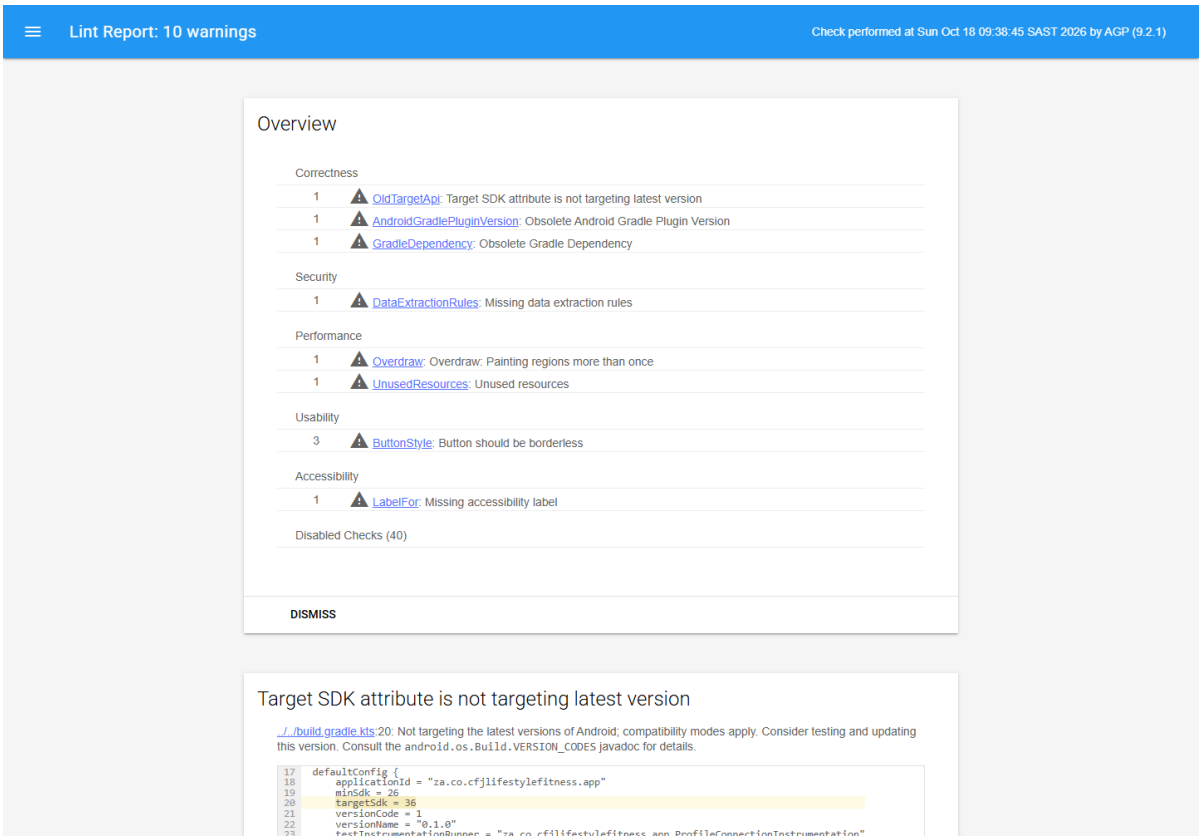


Figure 11. Android lint report: 0 errors and 10 warnings at the 18 October run; the 19 October build reports 0 errors and 22 warnings. Warnings are tracked as polish items, not failures.

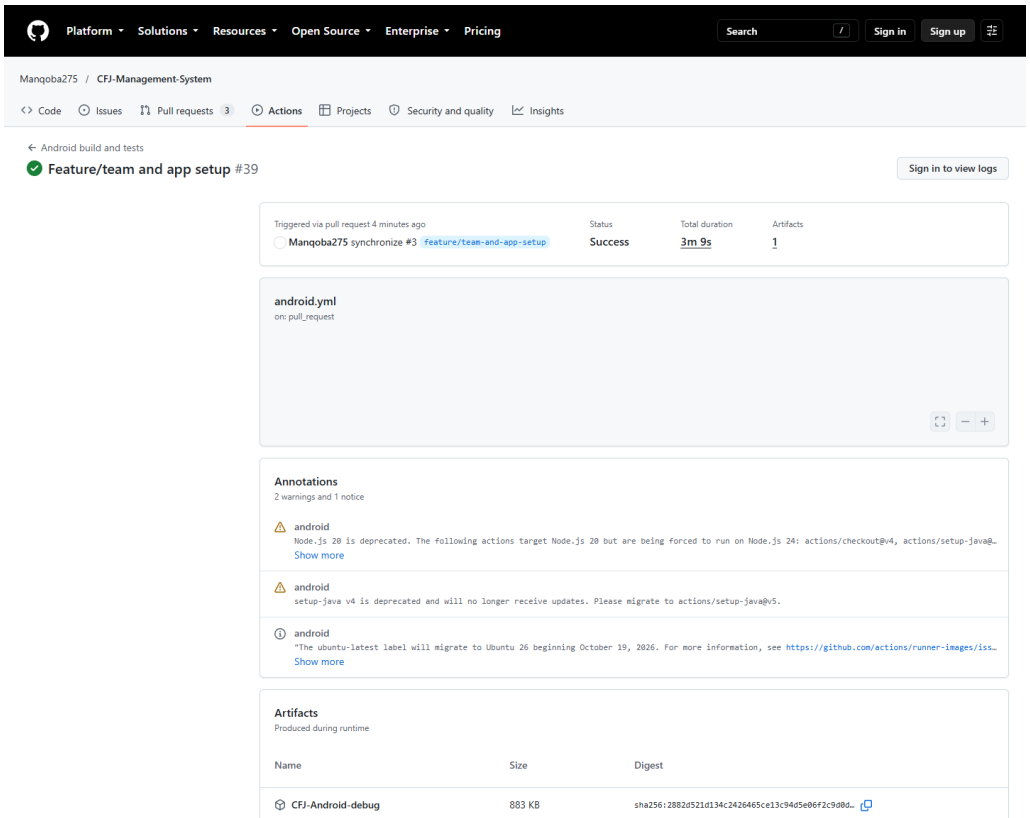


Figure 12. GitHub Actions: Android build and tests workflow succeeded on the submission commit 12c0312 (run 39309331362).

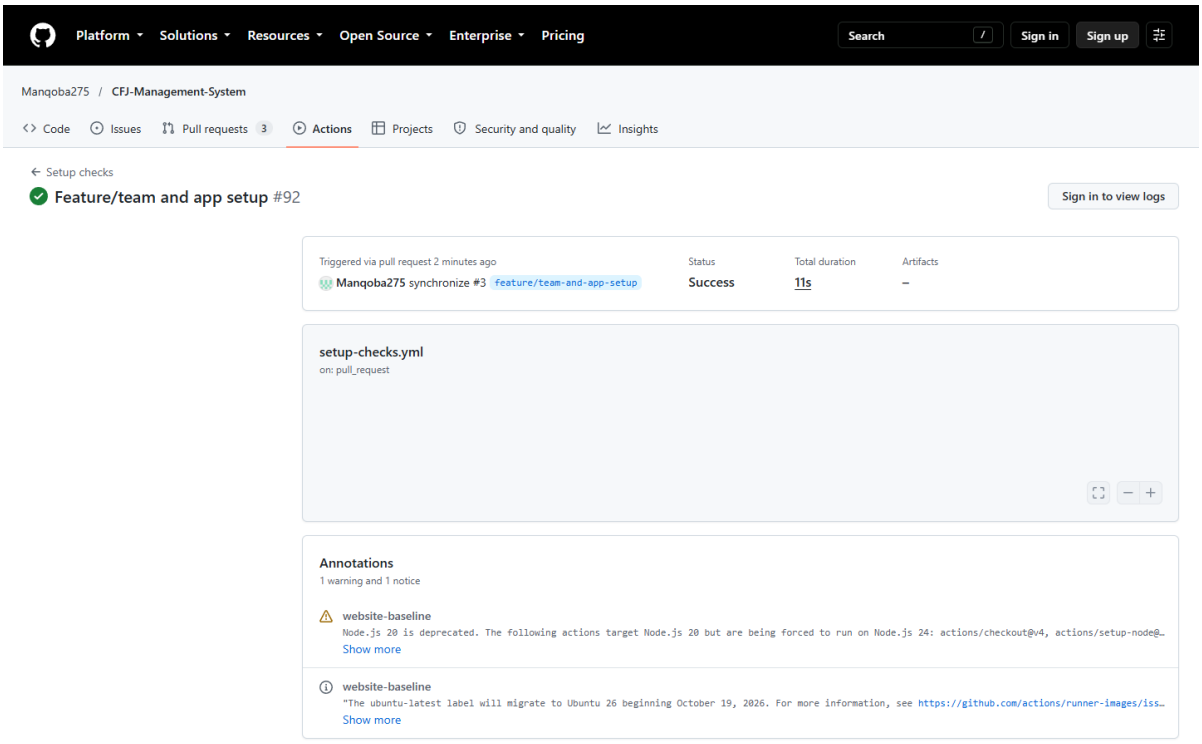


Figure 13. GitHub Actions: setup checks workflow succeeded, running npm test and the website check (run 39309331410).

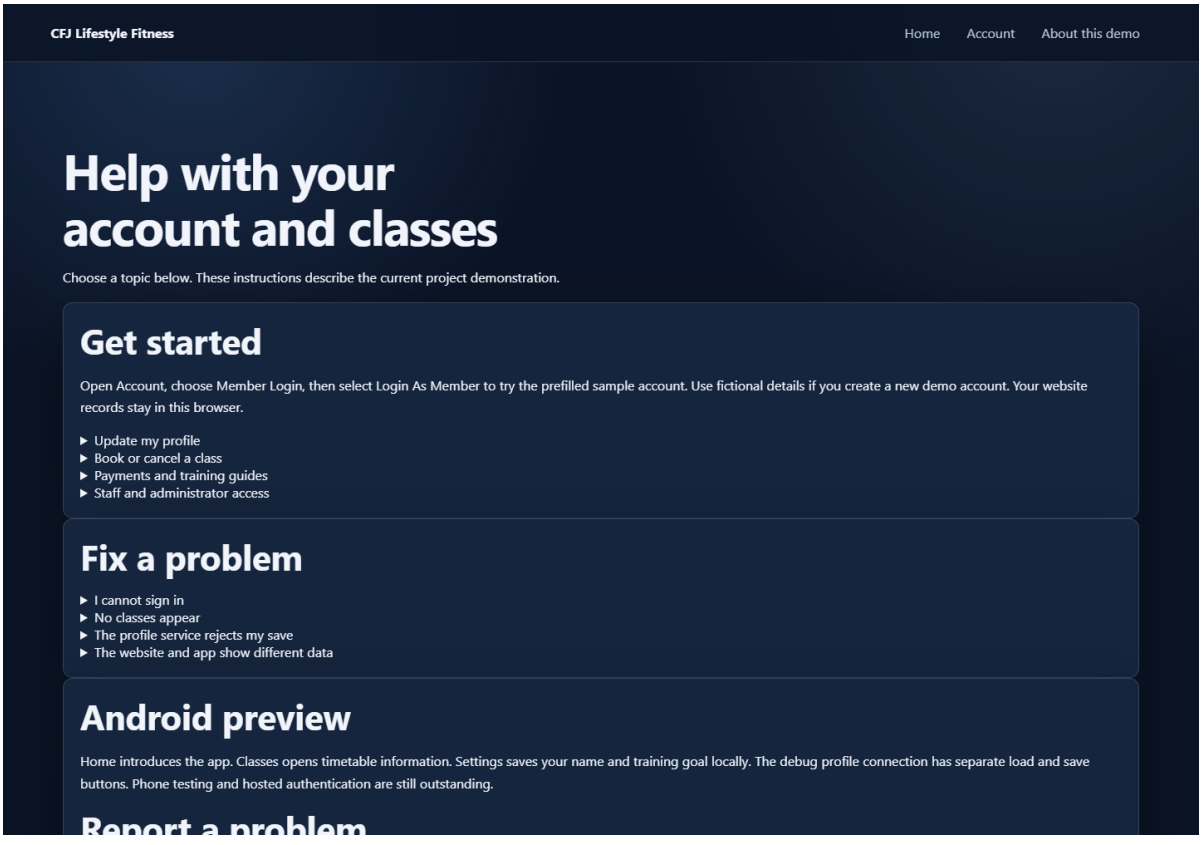


Figure 14. Help system page, live on Render. Part of the user-support requirement and linked from the footer of every page.

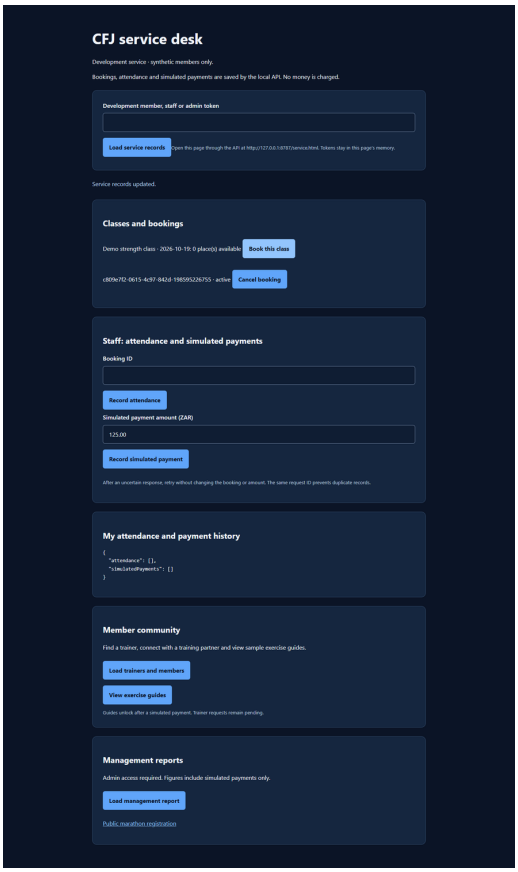


Figure 15. Booking and API integration evidence from the browser: a booking action against the local API, with the resulting state visible to a second client (19 October 2026).

6 GitHub repository and GitHub Actions

6.1 Repository

The project is a single public GitHub repository: https://github.com/Manqoba275/CFJ-Management-System. It contains the website (website/), the Kotlin Android app (mobile-app/), the Node.js API (backend/), the Oracle migrations and verifiers (database/, scripts/), the user and design documentation (docs/) and the CI workflows (.github/workflows/). The README at the repository root is the entry point, and CONTRIBUTING.md records the branch, commit and pull-request rules the team followed.

6.2 Branching and pull requests

Work was delivered through pull requests against main. PRs #1, #3, #4 and #5 are merged and brought in the project structure and documentation, the schema and seeds, and the testing evidence. PR #6 carries the Firebase account flows, the Android account screen and the Oracle identity mapping; it is complete and mergeable but was still open at the time of writing, so this report cites the implementation commit 12c0312 and the tag task2-submission rather than relying on the default branch alone.

6.3 Workflows

android.yml runs on changes under mobile-app/: it executes the unit tests and lint, assembles the debug and androidTest APKs, checks that no signing secret is committed, and uploads the APK. setup-checks.yml runs on every push: it installs Node dependencies, runs npm test (51 tests) and npm run check (17 pages). Both workflows need no manual input, which is why the marker can re-run them from the Actions tab.

6.4 Commit history

The repository has a real, incremental history rather than a single import: more than 190 workflow runs have executed, and Section 7 counts the commits per contributor from the full history across all refs.

7 Team contribution

7.1 Method

Contribution is measured from git history across all refs (git log --all with per-author counts), not from self-reported logs. Merge commits and documentation-only commits are included, because documentation and evidence form part of the deliverable. Two git identities belong to the same member: the local identity used on the project computer and the GitHub account identity.

7.2 Contribution matrix

Table 8. Commit contribution by member, counted from all git refs on 19 October 2026.

Member	Git identity	Commits	Main areas
Sanele Manqoba Mazibuko	mtung@KALIBER; Manqoba275	69	Architecture, API, Firebase gate, Android app, CI, report and evidence
Tshifhiwa Thamagane	Chiefie / Tshifhiwa Thamagane	14	Oracle schema, seeds, constraints, shared DTO contracts
Nyito Ramudzuli	nyito	13	Business analysis, requirements and documentation branches
Nonhlanhla Chirwa	Nonhlanhla Chirwa	9	Test plan, test cases, acceptance report, video plan
Total	-	105	-

7.3 Notes on attribution

Two points are stated for transparency. First, the 69 commits attributed to Manqoba are split across a local git identity and the GitHub identity; they are one member, but automated tooling that matches only one identity will under-count the contribution. Second, Nyito's thirteen commits are on documentation branches that are not yet merged into the default branch, so they do not appear in a default-branch view of the repository. Merging those branches is follow-up work; the commits themselves are real and are counted here.

Code review is thin: one formal pull-request review exists in the history (PR #3, changes requested). The team's review practice relied on pull requests and shared test evidence rather than recorded approvals, which is a process weakness rather than a technical one.

8 Limitations and unproven items

The following items are not proven by evidence in this pack. They are listed so that the marker can weigh them directly; none of them is presented elsewhere in the report as complete.

Table 9. Items that are not proven, and their effect on the submission.

Item	What is missing	Effect
Phone installation record	No adb install output, device model, Android version or logcat tied to the APK hash	The running build on the phone is demonstrated by screenshot, not by an install record
Repeated sign-in cycle	Register, sign out and sign in again not captured on the phone	Only one verified signed-in state is evidenced
Google SSO end-to-end	No completed Google consent and callback; the demo phone has no Play services	SSO is implemented and configured, not proven working end-to-end
Live Firebase to Oracle request from the app	No captured request with a real ID token resolving to a member row	The API path is tested server-side; the device-to-database demonstration is outstanding
Hosted API and hosted database	API and Oracle run on the project computer, bound to loopback	The brief's online-hosted demonstration requirement is not met
Database re-run by the marker	Local Oracle is not reachable after the session	Verification rests on stored artefacts and reproducible scripts
Management roles in the schema	No staff/admin table; roles are server-side development identities	Management features are API-level, not modelled as tables
Organisation feedback	No feedback record from CFJ exists	The 'updated based on feedback' rubric line is unproven
Meetings and Kanban evidence	Meeting minutes templates and an Azure DevOps plan exist; no attendance, minutes or board export	Process evidence is weak
Demonstration video	Not recorded	Ten marks with no evidence

Two design notes belong here as well. The active schema models members, operations and community, but not staff administration; reports are computed from committed records instead. And the website's demo member flows (registration, profile, bookings) still use browser localStorage with simulated payments, while the Firebase account page and the account screen are the API-backed identity path. The two are separate so that a demonstration cannot confuse a browser demo record with a database record.

9 Demonstration video

The rubric requires a professional, narrated video recorded on a physical phone that demonstrates every feature and shows what is stored in the online authentication service, the API and the database. The recording is the final outstanding deliverable.

VIDEO LINK: VIDEO_LINK_HERE

The video must show, in this order, with voice-over throughout:

1. The app opening on the physical phone, with the device visible and the app name and version on screen.
2. Registration with a new email address, the verification email, and the verified state shown in the app.
3. Sign out and sign in again, proving a repeat login, then the Google SSO flow on a device with Play services (or an explicit statement that the device cannot).
4. The settings screen: change the display name and training goal, save, reopen and confirm persistence.
5. The training dashboard: exercise guide, set completion and the rest timer.
6. A booking action in the app and the same record visible in the website, proving shared API and database state.
7. The Firebase console showing the created account, the API response showing the same member, and an Oracle query showing the member row and its profile.
8. Invalid input handling: bad email, empty password and a capacity conflict, each rejected with a readable message and no crash.

Length target is six to eight minutes, one narrator, no passwords or tokens on screen. The video is uploaded unlisted and linked here and in the repository README.

10 Conclusion

The CFJ Management System has a working website on Render, a native Android app that builds and has run on a physical phone, a Firebase-backed identity layer, a tested Node.js API and a verified fourteen-table Oracle schema with 160 rows and 114 constraints. Automated testing and CI are real: 51 web and API tests, 16 Android unit tests, 21 schema checks, 15 operations checks and two green workflows on the implementation commit.

The submission is honest about what is not yet there: a hosted API and database, a recorded demonstration video, a complete Google SSO acceptance, an install record for the phone, refreshed Android documentation and organisation feedback. The first items in that list are the highest-value remaining work. Everything else in this report is supported by a committed artefact, a script that can be re-run, or a CI run that the marker can open.

Appendix A Official Task 2 assessment rubric

The official assessment sheet for Task 2 (module manual pages 35-38) is appended to this report as the final four pages of the PDF. It includes the group member and student number fields, the ten marking criteria with their mark bands, and the Part 2 total of 100. The group members and their student numbers are also listed in Table 10.

Table 10. Group members and student numbers.

Student name	Student number	Contribution focus
Sanele Manqoba Mazibuko	ST10444275	Architecture, API, Android, CI, report
Nonhlanhla Chirwa	ST10451192	Testing, acceptance, video plan
Tshifhiwa Thamagane	ST10441732	Database design, seeds, constraints
Nyito Ramudzuli	ST10450776	Requirements, business analysis, documentation

Appendix B Evidence index

Evidence is stored in the repository under docs/evidence/ and in the submission pack. Paths are relative to the repository root below.

Table 11. Evidence index: artefact, location and the claim it supports.

Artefact	Location	Supports
Schema verification (JSON)	docs/evidence/database-2026-10-18/schema-verification.json	14 tables, 160 rows, 21 checks
Live schema re-verification	docs/evidence/live-2026-10-19/oracle-schema.txt	Migrations 001-004 idempotent; constraints ENABLED and VALIDATED
Live operations verification	docs/evidence/live-2026-10-19/oracle-operations.txt	15 checks PASS, two HTTP servers
Row counts	docs/evidence/live-2026-10-19/row-counts.txt	20/20/10 x 12, total 160, PASS
Seed precondition	docs/evidence/live-2026-10-19/seed-precondition-readonly.txt	Member 9001 bound to an external Firebase UID
Test run	docs/evidence/live-2026-10-19/npm-test.txt	51 tests pass, 0 fail
Auth-boundary tests	docs/evidence/firebase-oracle-2026-10-19/auth-boundary-tests.txt	6 of 6 authentication-boundary tests pass
Android build log	docs/evidence/local-oracle-phone-2026-10-19/android-build.txt	BUILD SUCCESSFUL, unit tests and lint included
Android sign-in screenshot	03-Evidence/phone-verified-sign-in.png	App ran on a physical phone; verified signed-in state
Website and help captures	03-Evidence/CFJ WIL Evidence 2026-10-18/	Live site, help page, invalid input, test and CI screens
CI runs	https://github.com/Manqoba275/CFJ-Management-System/actions	Android 39309331362 and setup 39309331410, both success on 12c0312
APK	02-Android/CFJ-Fitness-debug.apk	8,445,659 bytes; SHA-256 90A810FF9248974A06E3D6BA570354E962912B398FB6F7EEBF9067835A725E76

End of report.

ASSESSMENT SHEET (TASK 2 - MARKING RUBRIC)

Please note: Attach this rubric to your work when you submit it online.

MODULE NAME:	MODULE CODE:
WORK INTEGRATED LEARNING	XISD6329\w

GROUP MEMBERS
STUDENT NAME:
STUDENT NUMBER: STUDENT NAME:
STUDENT NUMBER: STUDENT NAME:
STUDENT NUMBER:

© The Independent Institute of Education (Pty) Ltd 2026

TASK 2					
Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard	Feedback
The app runs on a mobile device.	The app doesn't run at all. The app runs, but lots of crashes happen.	The app runs with one or two errors or crashes. Runs on the emulator	The app runs with very infrequent errors or crashes.	The app runs without errors.	
[10 Marks]	0 – 4 Marks	5 Marks	6 – 7 Marks	8 – 10 Marks	
Feature: sign in [10 Marks]	Feature not included. Feature implemented but very buggy.	The feature is working mostly with some bugs.	The feature is working with only minor bugs.	Feature excellently implemented.	
	0 – 4 Marks	5 Marks	6 – 7 Marks	8 – 10 Marks	
Feature: Settings menu that makes sense for the application	Feature not included. Feature implemented but very buggy.	Feature is working mostly with some bugs.	Feature is working with only minor bugs.	Feature excellently implemented.	
[10 Marks]	0 – 4 Marks	5 Marks	6 – 7 Marks	8 – 10 Marks	
Creation/use of the API	API not included. API was created, but very buggy.	API is working mostly with some bugs.	API is working with only minor bugs.	API excellently created/used	
[10 Marks]	0 – 4 Marks	5 Marks	6 – 7 Marks	8 – 10 Marks	

© The Independent Institute of Education (Pty) Ltd 2026

Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard	Feedback
Integration of your API in your app [10 Marks]	Feature not included. Feature implemented, but has many errors	The feature is working mostly with some bugs.	The feature is working with only minor bugs.	The feature is excellently integrated.	
	0 – 4 Marks	5 Marks	6 – 7 Marks	8 – 10 Marks	
Database Structure & Table Design [15 Marks]	Database non-existent OR major structural flaws. Tables are missing or incorrectly defined	Basic tables created but several structural errors (missing primary keys, incorrect data types)	Tables mostly correct. Minor structural issues (e.g., minor datatype mismatches, weak naming conventions).	Well-structured relational database. Clear naming conventions. Correct data types. All tables are logically designed.	
	0 – 6 Marks	7-9 Marks	10 – 12 Marks	13– 15 Marks	
User interface [10 Marks]	Not user-friendly, confusing, and illogical. Somewhat user-friendly, with lots of mistakes in the design.	Mostly user-friendly, with some mistakes in the documentation	Mostly user-friendly, with some attempt made at making the capture process easier for the user.	Excellent and user-friendly. Completely consistent use of layout, fonts, and colours.	
	0 – 4 Marks	5 Marks	6 – 7 Marks	8 – 10 Marks	

© The Independent Institute of Education (Pty) Ltd 2026

Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard	Feedback
User documentation [10 Marks]	Not user-friendly, confusing, and illogical. Somewhat user-friendly, with lots of mistakes in the documentation	Mostly user-friendly, with some mistakes in the design. Somewhat consistent use of layout, fonts, and colours.	Mostly user-friendly, with some attempt made at making the capture process easier for the user. Mostly consistent use of layout, fonts, and colours.	Clear, concise, and professional documentation. All interface elements are clearly labelled.	
	0 – 4 Marks	5 Marks	6 – 7 Marks	8 – 10 Marks	
Github, Readme and automated testing [10 Marks]	Zip file added or file upload used for GitHub.	Multiple commits and unit testing are not working. No readme file.	Automated testing includes but is not working correctly. Somewhat readme file created.	Readme file created and implemented correctly. Automated testing is in place. Multiple commits.	
	0 – 4 Marks	5 Marks	6 – 7 Marks	8 – 10 Marks	
Demonstration video [10 Marks]	No demonstration video included. Informal and unprepared or not showing all the features or no voice-over included.	It is not completely professional, but all features are demonstrated.	Mostly professional with some issues, but all features were demonstrated.	Fully professional video showing all the required features in detail.	
	0 – 4 Marks	5 Marks	6 – 7 Marks	8 – 10 Marks	Part 2 Total /100

© The Independent Institute of Education (Pty) Ltd 2026